

Práctica de Laboratorio

Jardín Virtual con WebXR utilizando Hit Test

Introducción

En esta práctica se desarrolla una aplicación de Realidad Aumentada utilizando la API WebXR. El programa emplea la técnica denominada **Hit Test**, cuyo objetivo es detectar superficies reales utilizando la cámara y los sensores del dispositivo móvil.

Cuando el usuario inicia la sesión de Realidad Aumentada, el programa analiza continuamente el entorno buscando superficies horizontales. Una vez localizada una superficie válida, aparece una retícula que indica el lugar donde podrá colocarse un objeto virtual. Cada vez que el usuario toca la pantalla, el sistema selecciona aleatoriamente uno de siete modelos tridimensionales de flores y lo coloca sobre la superficie detectada.

A diferencia del ejemplo original proporcionado por Immersive Web, donde únicamente existe una flor que cambia de posición, esta versión permite construir un pequeño jardín virtual compuesto por múltiples flores diferentes distribuidas dentro del espacio físico.

Estructura general del programa

El archivo **index.html** contiene tres partes principales.

La primera corresponde al documento HTML encargado de construir la interfaz mínima de la aplicación.

La segunda importa las bibliotecas necesarias para trabajar con WebXR, WebGL y modelos tridimensionales.

La tercera implementa toda la lógica del programa mediante funciones escritas en JavaScript.

Importación de bibliotecas

El programa comienza importando varias clases mediante la instrucción **import**.

```
import {WebXRButton} from './js/util/webxr-button.js';
```

Esta clase crea automáticamente el botón que permitirá iniciar y finalizar la sesión de Realidad Aumentada.

Posteriormente se importa la clase **Scene**, responsable de administrar todos los objetos que existirán dentro del mundo virtual.

```
import {Scene} from './js/render/scenes/scene.js';
```

También se importa la clase **Renderer**, encargada de dibujar todos los modelos tridimensionales utilizando WebGL.

```
import {Renderer, createWebGLContext} from './js/render/core/rendererer.js';
```

La clase **Node** representa un nodo dentro del grafo de escena. Todos los objetos tridimensionales son nodos.

```
import {Node} from './js/render/core/node.js';
```

Los modelos tridimensionales en formato GLTF son cargados mediante la clase **Gltf2Node**.

```
import {Gltf2Node} from './js/render/nodes/gltf2.js';
```

Finalmente se importan las clases necesarias para generar sombras y realizar operaciones matemáticas con vectores.

Variables globales

El programa declara varias variables globales que permanecerán disponibles durante toda la ejecución.

```
let xrButton = null;  
let xrRefSpace = null;  
let xrViewerSpace = null;  
let xrHitTestSource = null;
```

Estas variables almacenan la sesión XR, el espacio de referencia utilizado para colocar los objetos y la fuente encargada de realizar continuamente las pruebas de impacto sobre las superficies del mundo real.

Posteriormente se crean las variables relacionadas con el motor gráfico.

```
let gl = null;  
let renderer = null;  
let scene = new Scene();
```

La escena constituye el contenedor principal donde serán agregados todos los objetos virtuales.

Nodo base

Aunque en esta versión cada flor es creada dinámicamente, el programa conserva un nodo denominado **arObject**.

```
let arObject = new Node();  
arObject.visible = false;  
scene.addNode(arObject);
```

Este nodo permanece oculto y únicamente sirve como punto de referencia dentro de la escena.

Arreglo de modelos

Una de las modificaciones más importantes consiste en crear un arreglo que contiene las rutas de los siete modelos tridimensionales.

```
const flowerFiles = [  
  'Flor01.glTF',  
  'Flor02.glTF',  
  ...  
  'Flor07.glTF'  
];
```

Obsérvese que el arreglo no almacena los modelos, sino únicamente las direcciones donde se encuentran los archivos GLTF.

Cada posición del arreglo representa un modelo distinto.

Índice	Modelo
0	Flor01.glTF
1	Flor02.glTF
2	Flor03.glTF
3	Flor04.glTF
4	Flor05.glTF
5	Flor06.glTF
6	Flor07.glTF

Posteriormente el programa elegirá una de estas posiciones utilizando un número aleatorio.

Retícula

La retícula constituye una ayuda visual para el usuario.

```
let reticle = new Gltf2Node({  
  url:'media/gltf/reticle/reticle.glTF'  
});
```

Inicialmente permanece oculta.

```
reticle.visible = false;
```

Cuando el Hit Test detecta una superficie, la retícula aparece exactamente en el punto donde podrá colocarse una flor.

Límite de flores

Para evitar consumir demasiada memoria se establece un límite máximo de objetos.

```
const MAX_FLOWERS = 30;
```

Esto significa que nunca existirán más de treinta flores simultáneamente dentro de la escena.

Las flores colocadas por el usuario se almacenan en el arreglo

```
let flowers = [];
```

Cada elemento del arreglo contiene la referencia a una flor existente.

Función initXR()

Esta función constituye el punto de entrada del programa.

Su responsabilidad consiste en crear el botón **START AR**, comprobar si el navegador soporta WebXR y habilitar el botón únicamente cuando el dispositivo sea compatible.

Si el navegador no soporta Realidad Aumentada, el botón permanece deshabilitado.

Función onRequestSession()

Cuando el usuario presiona el botón START AR se ejecuta esta función.

```
navigator.xr.requestSession(...)
```

Aquí se solicita al navegador una sesión del tipo

```
immersive-ar
```

Además se indican dos características obligatorias:

- local
- hit-test

La primera permite trabajar con un sistema de coordenadas estable.

La segunda habilita la detección de superficies.

Función onSessionStarted()

Esta función realiza toda la configuración inicial de la sesión XR.

Primero crea el contexto WebGL compatible con WebXR.

Posteriormente construye el objeto `Renderer`.

Después solicita dos espacios de referencia.

viewer

Representa la posición del observador.

local

Representa el sistema de coordenadas donde existirán los objetos virtuales.

Finalmente crea el objeto denominado

`xrHitTestSource`

que realizará continuamente las pruebas de impacto sobre las superficies reales.

Función addARObjectAt()

Esta función constituye la principal modificación realizada al ejemplo original.

Su responsabilidad consiste en crear una nueva flor cada vez que el usuario toca la pantalla.

Primero crea un nodo vacío.

```
let newFlower = new Node();
```

Después selecciona un número aleatorio.

```
let randomIndex =  
Math.floor(Math.random()*flowerFiles.length);
```

La función **Math.random()** genera un número entre 0 y 1.

Al multiplicarlo por la longitud del arreglo se obtiene un número comprendido entre 0 y 6.

Finalmente **Math.floor()** elimina la parte decimal.

Por ejemplo

Valor generado	Resultado
0.13	0
2.89	2
6.71	6

Con este índice el programa conoce cuál de las siete flores deberá cargar.

```
let flower = new Gltf2Node({  
  url: flowerFiles[randomIndex]  
});
```

En lugar de reutilizar siempre el mismo modelo, cada toque crea una nueva instancia del modelo seleccionado.

Posteriormente el modelo se agrega al nodo.

```
newFlower.addNode(flower);
```

Después se crea una sombra.

```
let shadow = new DropShadowNode();
```

La sombra mejora considerablemente la sensación de profundidad y hace que la flor parezca apoyarse realmente sobre el suelo.

Finalmente el nuevo nodo recibe la matriz de transformación calculada por el Hit Test.

```
newFlower.matrix = matrix;
```

Gracias a esta instrucción la flor aparece exactamente donde estaba ubicada la retícula.

Administración de memoria

Una vez creada la flor, ésta se almacena dentro del arreglo.

```
flowers.push(newFlower);
```

La instrucción **push()** inserta el nuevo objeto al final del arreglo.

Posteriormente el programa verifica si se alcanzó el límite permitido.

```
if(flowers.length>MAX_FLOWERS)
```

Cuando existen más de treinta flores se ejecuta

```
flowers.shift();
```

La función **shift()** elimina el primer elemento del arreglo, es decir, la flor más antigua.

Posteriormente también desaparece de la escena mediante

```
scene.removeNode(oldFlower);
```

Este mecanismo implementa una estructura de datos **FIFO (First In, First Out)**.

La primera flor colocada es también la primera en eliminarse.

Gracias a esta estrategia el número de objetos permanece constante y la memoria utilizada por la aplicación nunca crece indefinidamente.

Función onSelect()

Esta función responde al toque realizado por el usuario.

Antes de colocar una flor verifica que exista una superficie válida.

```
if(!reticle.visible)
```

```
    return;
```

Si la retícula no es visible significa que todavía no se detectó ninguna superficie.

Cuando la retícula sí está visible simplemente llama a

```
addARObjectAt(reticle.matrix);
```

utilizando la posición exacta donde el Hit Test encontró el punto de impacto.

Función onXRFrame()

Esta función constituye el ciclo principal del programa.

Se ejecuta aproximadamente sesenta veces por segundo.

En cada iteración realiza las siguientes operaciones.

1. Obtiene la posición actual del usuario.
2. Ejecuta el Hit Test.
3. Comprueba si existe una superficie válida.
4. Mueve la retícula al punto detectado.
5. Renderiza toda la escena.
6. Solicita el siguiente cuadro de animación.

Este ciclo continúa ejecutándose mientras permanezca activa la sesión de Realidad Aumentada.

Conclusiones

Esta versión representa una evolución importante del ejemplo oficial de WebXR. Mientras que el programa original únicamente permitía mover una sola flor sobre diferentes superficies, la nueva implementación crea un jardín virtual compuesto por múltiples flores elegidas aleatoriamente.

Desde el punto de vista de la programación, esta práctica introduce conceptos fundamentales como la carga dinámica de modelos GLTF, el uso de arreglos para administrar recursos, la generación de números aleatorios, la creación dinámica de objetos tridimensionales y la implementación de una estructura FIFO para controlar el consumo de memoria. El resultado es una aplicación de Realidad Aumentada más completa, visualmente atractiva y suficientemente eficiente para ejecutarse en dispositivos móviles compatibles con WebXR.

Código QR

